

P04. Orchestrierung und verteilte Transaktionen

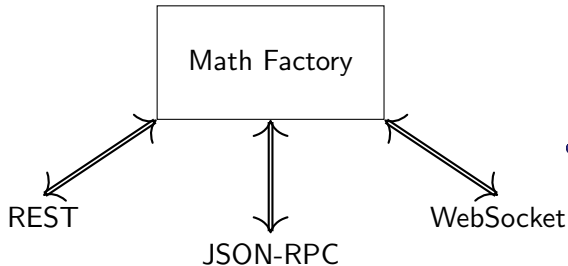
Bausteine Verteilter Systeme

SoSe2026

Math Factory

Aufgabe 2: JSON-RPC

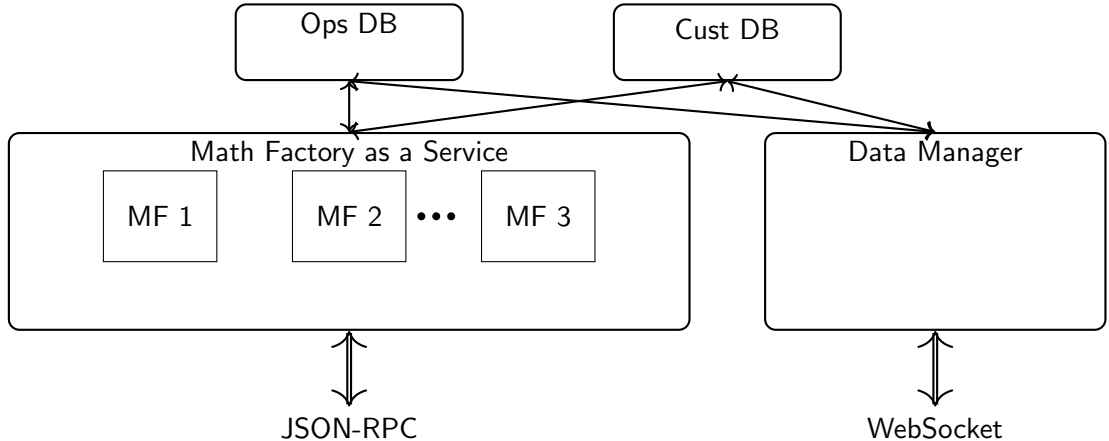
Aufgabe 2: Math-Factory – JSON-RPC Server Rechnungen auszulagern



- **JSON-RPC** um Mathe-Funktionen bereitzustellen
 - ▶ Addition
 - ▶ Subtraktion
 - ▶ ...
- **REST** um eine Admin-Schnittstelle zu bieten
 - ▶ Funktionen ein- oder auszuschalten bzw. hinzuzufügen oder zu entfernen
 - ▶ Rechenkosten einzustellen
- **WebSocket** als eine Schnittstelle um asynchrone und echtzeit Benachrichtigungen zu ermöglichen

Math Factory as a Service

Erweiterung der Math Factory



Math Factory as a Service

Komponenten

- **Math Factory** – JSON-RPC Server sind als Service Instanzen (MF 1, MF 2, ...)
 - ▶ Muss nach Bedarf hoch- oder runterskaliert werden
 - ▶ Requests sollen unter den Instanzen verteilt werden
 - ▶ Der Port für den JSON-RPC Server soll nach außen sichtbar sein
- **Data Manager** – Datenbankzugriff und Benachrichtigungen
 - ▶ Anwendungen die verursachten Kosten abfragen
 - ▶ Kosten-Thresholds für Benachrichtigungen setzen
 - ▶ Benachrichtigungen bekommen, wenn der Threshold überschritten wird

Math Factory as a Service

Komponenten

- **Ops DB** – Datenbank für Serverinstanz bezogene Daten
 - ▶ Welche Operation wurde durchgeführt
 - ▶ Was hat die Operation gekostet
 - ▶ Für welche Anwendung wurde die Operation durchgeführt
 - ▶ Server Instanz Statistiken
 - ▶ ...
- **Cust DB** – Datenbank für Anwendungsbezogene Daten
 - ▶ Welche Operation hat die Anwendung angefordert?
 - ▶ Wie viel hat es die Anwendung gekostet?
 - ▶ Auf welche Server Instanz wurde die Rechnung durchgeführt?
 - ▶ Statistiken für die Anwendung
 - ▶ ...

Math Factory as a Service

Docker Swarm zur Orchestrierung

- Docker Swarm bietet eine einfache Möglichkeit, Container Rechenknoten übergreifend zu orchestrieren. Aber der Swarm kann auch auf einem einzelnen Knoten betrieben werden.
- Die teilnehmenden Knoten sind in einem Docker Swarm Cluster organisiert
- Ähnlich zu Docker Compose können die Komponenten in einer YAML-Datei beschrieben werden, die dann auf den Knoten verteilt wird

Docker Swarm

```
$ docker swarm init
Swarm initialized: current node (69cdo3uzeqgjgeahimp6q09qb)
is now a manager.
$ docker swarm join --token <token> <ip>:<port>
$ docker node ls
```

Math Factory as a Service

Docker Swarm zur Orchestrierung

- Knoten können als **Manager** oder **Worker** konfiguriert werden
- Der Cluster kann auch während der Laufzeit verändert werden – Knoten können hinzugefügt oder entfernt werden
- Docker Swarm nutzt Virtual eXtensible LAN (VXLAN) um Container über Knoten hinweg zu verbinden
 - ▶ VXLAN ist ein Overlay-Protokoll, das Layer-2-Netzwerke über Layer-3-Netzwerke erstreckt
 - ▶ VXLAN kapselt Ethernet-Frames in UDP-Pakete
 - ▶ Per Default Ports 4789 und 7946 wird beim Swarm genutzt. Diese Ports müssen in der Firewall geöffnet werden.
- Docker Swarm muss die Cluster Daten auch Konsistent halten. Hierzu wird **Raft**, ein verteiltes Konsensprotokoll, verwendet.

Math Factory as a Service

Docker Swarm zur Orchestrierung

Service Skalierung

```
$ docker service create --name math-factory \  
  --replicas 3 \  
  --network math-factory-net \  
  --publish mode=host,target=8080,published=8080 \  
  math-factory:latest  
$ docker service ls
```


Math Factory as a Service

Datenbanken

- Die Math Factory Instanzen und der Data Manager müssen auf die Datenbanken zugreifen bzw. Daten austauschen
- Modellieren Sie diese Austauschvorgänge als Transaktionen
 - ▶ Nutzen Sie die Notationen aus der Vorlesung
 - ▶ Zeichnen Sie die History einer möglichen Ausführung der Transaktionen mit der zusammengehörigen Serialisierungsgraph auf
 - ▶ Stellen Sie sicher, dass der Serialisierungsgraph keine Zyklen enthält
- Die Daten zwischen den Datenbanken müssen konsistent sein
- Daher wählen Sie eine Datenbank, die multi Node Transaktionen unterstützt